

Deep Dive — Module 1: PostgreSQL (Relational Database)

***Goal of this module:** understand every component of PostgreSQL that the Financial Document Similarity Finder uses — not just the syntax, but why each piece exists and how it shows up in the real project. By the end you should be able to read the project's `schema.sql`, explain every line, and predict how the backend talks to the database.*

Where PostgreSQL sits in the project: It is the **system of record** — the source of truth for facts you must never lose or corrupt: who the users are, what documents exist, the audit trail, and the alerts. The vector database (Qdrant) stores the *embeddings* used for similarity; PostgreSQL stores the *metadata and history*. If Qdrant were wiped you could re-embed; if PostgreSQL were corrupted you'd lose the financial record. That's why it must be ACID-compliant and carefully backed up.

Table of Contents

1. [Mental model: why a relational database](#)
2. [Tables, rows, columns, schema](#)
3. [Data types — every type the project uses, and why](#)
4. [DDL — defining structure](#)
5. [Constraints — the rules that protect your data](#)
6. [DML — inserting, reading, updating, deleting](#)
7. [Querying deeply: WHERE, ORDER BY, pagination, aggregates](#)
8. [Relationships & foreign keys](#)
9. [Joins](#)
10. [Indexes — making reads fast](#)
11. [Transactions & ACID](#)
12. [The soft-delete pattern](#)
13. [Append-only audit logs](#)
14. [Functions & triggers](#)

15. [JSONB — semi-structured data](#)
 16. [UUID vs serial keys](#)
 17. [pgvector — similarity inside SQL](#)
 18. [Connection pooling \(HikariCP\)](#)
 19. [Backup & recovery](#)
 20. [Security & least privilege](#)
 21. [Putting it together: the project's four tables](#)
 22. [Common pitfalls](#)
 23. [Practice exercises](#)
 24. [Self-check questions](#)
 25. [Glossary](#)
-

1. Mental model: why a relational database

A **relational database** organizes data into tables that *relate* to one another through shared keys. Three properties make PostgreSQL the right choice for the financial side of this system:

- **Structure & integrity.** A document *must* have a valid uploader, an invoice number *must* be unique, an amount *must* be a number. The database enforces these rules so bad data can't get in — even if the application code has a bug.
- **ACID transactions.** When you upload a document you write *both* the document row *and* an audit row. Either both happen or neither does. You can never end up with a document and no audit trail (a compliance disaster) or an audit entry pointing at a document that doesn't exist.
- **Mature querying.** SQL lets you ask complex questions ("all invoices from vendor X over \$10,000 uploaded last quarter") efficiently, with indexes keeping it fast at scale.

Contrast with Qdrant: Qdrant answers "which vectors are similar to this one?" PostgreSQL answers "what are the exact facts about these documents, and who did what when?" They are complementary, not competitors.

2. Tables, rows, columns, schema

A **table** models one kind of thing (users, documents, audit logs, alerts). A **row** (or *record/tuple*) is one instance. A **column** (or *field/attribute*) is one property, and every column has a fixed **data type**.

documents table

id (UUID)	filename (TEXT)	vendor (TEXT)	total_amount	← columns
a3f9c1b2.	invoice_2048.pdf	ABC Supplies	1250.75	← a row
d7e2a4f1.	receipt_991.pdf	ABC Supplies	1250.75	← another row

A **schema** (in the structural sense) is the full set of table definitions — the "shape" of your data. The project's schema lives in `schema.sql` and is loaded when the database container starts (Testcontainers uses `.withInitScript("schema.sql")` for integration tests). PostgreSQL also has a *namespace* concept also called a "schema" (e.g. `public`); don't confuse the two — most small projects just use the default `public` namespace.

3. Data types — every type the project uses, and why

Choosing the right type is a *correctness and performance* decision, not a formality. Here is every type in the project schema with the reasoning:

Type	Used for	Why this type
UUID	All primary keys (<code>id</code>)	Globally unique, unguessable, generatable by the app <i>or</i> the DB. Safe to expose in URLs/APIs; no sequential leakage of "how many documents exist."
TEXT	<code>filename</code> , <code>vendor</code> , <code>username</code> , <code>message</code>	Variable-length string with no length ceiling. In PostgreSQL <code>TEXT</code> and <code>VARCHAR</code> perform identically; <code>TEXT</code> avoids arbitrary length limits.
VARCHAR(n)	(alt) bounded strings	Same as <code>TEXT</code> but enforces a max length — use when a real business limit exists.
NUMERIC(15,2)	<code>total_amount</code>	Exact decimal: 15 total digits, 2 after the point. <i>Never</i> use <code>FLOAT</code> / <code>DOUBLE</code> for money — binary floats can't represent 0.10 exactly and you get rounding errors in financial totals.
CHAR(3)	<code>currency</code>	Fixed-length code, always exactly 3 chars (<code>USD</code> , <code>EUR</code> , <code>INR</code>).
BOOLEAN	<code>active</code> , <code>ocr_used</code>	true/false flags.
DATE	<code>invoice_date</code> , <code>retention_expires</code>	Calendar date, no time component.
TIMESTAMPTZ	<code>created_at</code> , <code>uploaded_at</code>	Timestamp with time zone — stores an absolute instant so logs are unambiguous across zones. Prefer <code>TIMESTAMPTZ</code> over <code>TIMESTAMP</code> for audit data.
INT	<code>chunks_count</code>	Whole numbers.
JSONB	audit <code>detail</code>	Stored, indexed , binary JSON. Holds flexible structured detail (e.g. <code>{"before": .{. }, "after": .{. } }</code>) without a rigid column per field.
INET	<code>ip_address</code>	Native IP-address type (validates format, supports network operators).
UUID[]	<code>document_ids</code> in alerts	An array of UUIDs — one alert can reference several documents (e.g. a duplicate pair).
VECTOR(384)	optional <code>embedding</code>	From the pgvector extension; 384-dimensional float vector for SQL-side similarity (see §17).

Key takeaway: money → `NUMERIC` , time → `TIMESTAMPTZ` , identity → `UUID` , flexible structure → `JSONB` . Picking wrong here causes subtle, expensive bugs later.

4. DDL — defining structure

DDL (Data Definition Language) creates and changes *structure*: `CREATE` , `ALTER` , `DROP` .

```
CREATE EXTENSION IF NOT EXISTS vector;      -- enable pgvector before using VECTOR

CREATE TABLE documents (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  filename          TEXT NOT NULL,
  document_type     TEXT,                    -- invoice | receipt | statement | po
  vendor            TEXT,
  invoice_number    TEXT UNIQUE,
  invoice_date      DATE,
  total_amount      NUMERIC(15,2),
  currency          CHAR(3),
  status            TEXT DEFAULT 'INDEXING',
  ocr_used          BOOLEAN DEFAULT FALSE,
  chunks_count      INT,
  uploaded_by       UUID REFERENCES users(id),
  uploaded_at       TIMESTAMPTZ DEFAULT NOW(),
  embedding         VECTOR(384)
);
```

Changing structure later:

```
ALTER TABLE documents ADD COLUMN department TEXT;      -- add a column
ALTER TABLE documents ALTER COLUMN currency SET NOT NULL; -- tighten a rule
ALTER TABLE documents RENAME COLUMN vendor TO supplier; -- rename
DROP TABLE documents;                                -- remove the whole table (careful!)
```

Why `IF NOT EXISTS` on the extension? It makes the script idempotent — running it twice doesn't error. Init scripts should always be safe to re-run.

5. Constraints — the rules that protect your data

Constraints are guarantees the database enforces on every write. They are your last line of defense against bad data.

Constraint	Meaning	Project example
PRIMARY KEY	Unique + not null; identifies the row	<code>id UUID PRIMARY KEY</code>
NOT NULL	Value is required	<code>filename TEXT NOT NULL</code>
UNIQUE	No two rows share this value	<code>invoice_number TEXT UNIQUE</code> — blocks indexing the <i>same invoice number</i> twice
DEFAULT	Auto-fill when omitted	<code>status TEXT DEFAULT 'INDEXING'</code>
REFERENCES (foreign key)	Value must exist in another table	<code>uploaded_by UUID REFERENCES users(id)</code>
CHECK	Custom boolean rule	<code>CHECK (total_amount ≥ 0)</code>

A `CHECK` example you might add for safety:

```
ALTER TABLE documents
  ADD CONSTRAINT chk_amount_nonneg CHECK (total_amount ≥ 0),
  ADD CONSTRAINT chk_status CHECK (status IN ('INDEXING','INDEXED','FAILED','DELETED'));
```

Why this matters for finance: the `UNIQUE` on `invoice_number` is part of duplicate-invoice protection at the database layer — even if two requests race, the second insert of the same invoice number fails cleanly rather than creating a silent duplicate payment risk.

6. DML — inserting, reading, updating, deleting

DML (Data Manipulation Language) reads and writes *data*: `INSERT` , `SELECT` , `UPDATE` , `DELETE` .

```
-- INSERT: list columns explicitly (resilient to schema changes)
INSERT INTO documents (filename, vendor, total_amount, currency)
VALUES ('invoice_acme_may2026.pdf', 'Acme Corp', 12500.00, 'USD');

-- INSERT and get the generated id back
INSERT INTO documents (filename, vendor) VALUES ('receipt.pdf', 'Acme Corp')
RETURNING id;

-- UPDATE a specific row (ALWAYS include a WHERE clause)
UPDATE documents SET status = 'INDEXED' WHERE id = 'a3f9c1b2-.';

-- DELETE a specific row (this project rarely hard-deletes – see §12)
DELETE FROM documents WHERE id = 'a3f9c1b2-.';
```

***The most dangerous mistake in SQL:** an `UPDATE` or `DELETE` without a `WHERE` changes/erases **every row**. Always write the `WHERE` first, then the `SET` .*

7. Querying deeply

`SELECT` is where most of your time goes. Build it up in layers:

```

-- Choose columns
SELECT filename, vendor, total_amount FROM documents;

-- Filter rows
SELECT * FROM documents WHERE vendor = 'Acme Corp' AND total_amount > 1000;

-- Pattern match (case-insensitive with ILIKE)
SELECT * FROM documents WHERE vendor ILIKE 'acme%';

-- Range & set membership
SELECT * FROM documents
WHERE invoice_date BETWEEN '2026-01-01' AND '2026-03-31'
AND currency IN ('USD','EUR');

-- NULL checks (use IS NULL, never = NULL)
SELECT * FROM documents WHERE deleted_at IS NULL;

-- Sort, newest first
SELECT * FROM documents ORDER BY uploaded_at DESC;

-- Pagination (the GET /documents endpoint uses this)
SELECT * FROM documents ORDER BY uploaded_at DESC
LIMIT 20 OFFSET 40;           -- page 3 of size 20

```

Aggregates & grouping — the basis for the fraud-detection "mean amount per vendor" logic:

```

SELECT vendor,
       COUNT(*)      AS doc_count,
       AVG(total_amount) AS avg_amount,
       MAX(total_amount) AS max_amount
FROM documents
WHERE deleted_at IS NULL
GROUP BY vendor
HAVING COUNT(*) > 5;           -- HAVING filters AFTER grouping; WHERE filters BEFORE

```

Filter order rule: `WHERE` runs before grouping (on raw rows); `HAVING` runs after grouping (on aggregated groups). Mixing these up is a classic bug.

8. Relationships & foreign keys

A **foreign key** makes a column point to a row in another table and *enforces* that the target exists.

```
CREATE TABLE documents (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  filename    TEXT NOT NULL,  
  uploaded_by UUID REFERENCES users(id) -- FK → users.id  
);
```

Relationship shapes:

- **One-to-many:** one user → many documents (the common case here).
- **One-to-one:** rare; e.g. a user → one profile row.
- **Many-to-many:** modeled with a *join table*. The project sidesteps this for alerts by using a `UUID[]` array column instead of a separate `alert_documents` table — simpler when you don't need to query the link heavily.

Referential actions decide what happens when a referenced row is removed:

```
uploaded_by UUID REFERENCES users(id) ON DELETE RESTRICT -- block deleting a user who has docum  
-- or ON DELETE SET NULL → keep the document, blank the uploader  
-- or ON DELETE CASCADE → delete the user's documents too (dangerous for financial records!)
```

For audit/financial integrity this project favors `RESTRICT` or soft-deletes over `CASCADE` — you almost never want financial rows silently vanishing.

9. Joins

A **join** combines rows from multiple tables using a matching condition.

```
-- INNER JOIN: only rows that match in BOTH tables
SELECT d.filename, d.total_amount, u.username AS uploaded_by_name
FROM documents d
JOIN users u ON d.uploaded_by = u.id;

-- LEFT JOIN: ALL documents, even if the uploader is missing (uploader columns become NULL)
SELECT d.filename, u.username
FROM documents d
LEFT JOIN users u ON d.uploaded_by = u.id;
```

Join type	Keeps
INNER JOIN	Only matching rows on both sides
LEFT JOIN	All left rows; right side NULL if no match
RIGHT JOIN	All right rows; left side NULL if no match
FULL JOIN	All rows from both sides

Aliases (`d` , `u`) keep queries readable and are required when two tables share a column name. **Practical tip:** for an audit screen joining `audit_logs` to `users` , a `LEFT JOIN` is safer — you still want to show the log entry even if the user record was later removed.

10. Indexes — making reads fast

Without an index, a filtered query does a **sequential scan** — reads every row. An index is a sorted side-structure (default **B-tree**) that lets PostgreSQL jump straight to matching rows, like a book's index.

```

CREATE INDEX idx_documents_vendor      ON documents(vendor);
CREATE INDEX idx_documents_uploaded_at ON documents(uploaded_at);
CREATE INDEX idx_audit_user            ON audit_logs(user_id);

-- Composite index: speeds up queries filtering/sorting on BOTH columns, in this order
CREATE INDEX idx_documents_vendor_date ON documents(vendor, invoice_date);

-- Partial index: only index the "live" rows the app actually queries
CREATE INDEX idx_documents_active ON documents(uploaded_at) WHERE deleted_at IS NULL;

-- GIN index for JSONB containment queries
CREATE INDEX idx_audit_detail ON audit_logs USING GIN (detail);

```

Inspect whether an index is used:

```

EXPLAIN ANALYZE
SELECT * FROM documents WHERE vendor = 'Acme Corp';
-- Look for "Index Scan using idx_documents_vendor" (good) vs "Seq Scan" (no index used)

```

Costs & rules of thumb:

- Indexes speed up reads but *slow down writes* (every insert/update must also update the index) and use disk.
- Index the columns you **filter, join, or sort** on frequently. Don't index everything.
- A **UNIQUE** constraint automatically creates an index (e.g. `invoice_number`).

11. Transactions & ACID

A **transaction** groups statements so they all succeed or all fail. This is the heart of financial integrity.

```

BEGIN;
  INSERT INTO documents (filename, vendor, status)
  VALUES ('invoice.pdf', 'Acme Corp', 'INDEXED');

  INSERT INTO audit_logs (user_id, action, document_id, outcome)
  VALUES ('usr-0042', 'DOCUMENT_UPLOAD', (SELECT id FROM documents WHERE filename='invoice.pdf'), 'SUCCESS');
COMMIT;  -- both saved together; if anything fails, ROLLBACK undoes both

```

ACID is the guarantee set:

Letter	Property	What it means here
A	Atomicity	All-or-nothing: never a document without its audit row
C	Consistency	Constraints always hold before & after the transaction
I	Isolation	Concurrent uploads don't corrupt each other's reads/writes
D	Durability	Once <code>COMMIT</code> returns, the data survives a crash (written to disk/WAL)

Isolation levels trade safety vs concurrency: `READ COMMITTED` (PostgreSQL default), `REPEATABLE READ`, `SERIALIZABLE` (strictest). For most of this app the default is fine; use `SERIALIZABLE` for sensitive "check-then-insert" duplicate logic if you need bullet-proof correctness under heavy concurrency.

In the app: Spring's `@Transactional` annotation wraps a service method in `BEGIN/COMMIT` automatically. The `DocumentService.upload(. )` method writes the document and audit log inside one transaction so partial writes can't happen.

12. The soft-delete pattern

This project **does not hard-delete** documents in normal operation. Instead it stamps a `deleted_at` column.

```
-- "Delete" = mark, don't remove
UPDATE documents SET deleted_at = NOW() WHERE id = 'a3f9c1b2-.';

-- EVERY normal query must exclude deleted rows
SELECT * FROM documents WHERE deleted_at IS NULL;
```

Why: the row stays available for **audit, recovery, and retention policy** before a controlled purge. Note that *true* GDPR "Right to Erasure" (`DELETE /documents/{id}` , admin-only) is the exception — it *does* hard-delete the row and purge Qdrant vectors. So the system has two distinct "delete" concepts: soft-delete (everyday) and erasure (compliance).

***Pitfall:** forgetting `WHERE deleted_at IS NULL` makes "deleted" documents reappear. A partial index (see §10) plus a database view of active documents helps avoid this.*

13. Append-only audit logs

The `audit_logs` table must be **tamper-proof**: you can insert, but never modify or remove. This is enforced at the database privilege level, so even a compromised app account can't rewrite history.

```
-- The application connects as a limited role: app_user
REVOKE UPDATE, DELETE ON audit_logs FROM app_user;

-- Now an attempted tamper fails:
DELETE FROM audit_logs WHERE id = '!.';
-- ERROR: permission denied for table audit_logs
```

This pairs with the compliance requirement that audit records are immutable and retained for years (see the PRD's compliance section). The app *adds* audit rows on login, upload, search, delete, and alert actions, and nothing — not even an admin through the app — can erase them.

14. Functions & triggers

A **function** is stored, reusable logic. A **trigger** runs a function automatically on `INSERT` / `UPDATE` / `DELETE`. The project uses one to keep `updated_at` current without trusting the app to remember.

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW(); -- NEW is the row about to be written
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_documents_updated_at
    BEFORE UPDATE ON documents
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

- `BEFORE UPDATE` → runs before the row is saved, so it can modify `NEW`.
- `FOR EACH ROW` → fires once per affected row (vs `FOR EACH STATEMENT`).

- `plpgsql` → PostgreSQL's procedural language.

Why a trigger instead of app code? *It's guaranteed — every update path (app, manual SQL, migration script) gets a correct timestamp. Logic that must always hold belongs in the database.*

15. JSONB — semi-structured data

`JSONB` stores JSON in an efficient, indexable binary form. The audit `detail` column uses it to capture flexible context that doesn't deserve its own column.

```
INSERT INTO audit_logs (user_id, action, detail)
VALUES ('usr-0042', 'DOCUMENT_UPDATE',
       '{"before": {"vendor": "Acme"}, "after": {"vendor": "Acme Corp"}}');

-- Query inside the JSON
SELECT * FROM audit_logs WHERE detail->> 'action_source' = 'web';    -->> returns text
SELECT * FROM audit_logs WHERE detail-> 'after'->> 'vendor' = 'Acme Corp';

-- Containment query (fast with a GIN index)
SELECT * FROM audit_logs WHERE detail @> '{"after": {"vendor": "Acme Corp"}}';
```

- `→` returns a JSON value; `→>` returns text.
- `@>` means "contains"; pair it with a `GIN` index for speed.

When to use JSONB vs columns: *put queryable, structured, always-present fields in real columns (types + constraints + indexes). Put variable, occasional, free-form detail in `JSONB`. Don't dump everything into `JSONB` — you lose type safety and constraints.*

16. UUID vs serial keys

Two ways to make primary keys:

	SERIAL / BIGSERIAL (auto-increment int)	UUID
Value	1, 2, 3...	a3f9c1b2-... random 128-bit
Guessable?	Yes (leaks counts, enables enumeration)	No
Generated by	Database only	App <i>or</i> database (<code>gen_random_uuid()</code>)
Size	4–8 bytes	16 bytes
Merge across systems	Collisions likely	Safe

This project chooses **UUID** for every table. Reasons: IDs appear in API responses and URLs (`/documents/{id}`), so they must not be guessable or leak how many records exist; and the app can generate an ID *before* the insert, simplifying the upload flow (it can reference the ID in Qdrant payloads and audit logs in the same transaction).

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
```

17. pgvector — similarity inside SQL

`pgvector` adds a `VECTOR(n)` type and similarity operators to PostgreSQL. In this project the *primary* vector store is Qdrant, but the schema keeps an **optional** `embedding` `VECTOR(384)` column for SQL-side similarity or hybrid queries.

```
CREATE EXTENSION IF NOT EXISTS vector;

-- distance operators:↔ Euclidean  <#> negative inner product  ⇔ cosine distance
SELECT id, filename, 1 - (embedding⇔ '[0.02, -0.11,...]') AS cosine_similarity
FROM documents
WHERE deleted_at IS NULL
ORDER BY embedding⇔ '[0.02, -0.11,...]' -- nearest first
LIMIT 5;

-- approximate-nearest-neighbour index for speed at scale
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops);
```

Why keep both Qdrant and pgvector? Qdrant is purpose-built for large-scale ANN search with rich payload filtering and is the main engine. pgvector is a convenient fallback for smaller corpora or for running a similarity check inside the same SQL transaction as a metadata query. The PRD lists a possible upgrade to a 768-dim model — note you'd change `VECTOR(384)` to `VECTOR(768)` here and the Qdrant collection size.

18. Connection pooling (HikariCP)

Opening a fresh database connection per request is slow (TCP + auth + setup). A **connection pool** keeps a set of open connections and lends them out. Spring Boot uses **HikariCP** by default.

```
# application.yml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/financial_docs
    username: app_user
    password: ${DB_PASSWORD}          # from env var / secret, never hard-coded
    hikari:
      maximum-pool-size: 20           # match expected concurrency
      minimum-idle: 5
      connection-timeout: 30000       # ms to wait for a free connection
```

- **Pool size** must align with PostgreSQL's `max_connections` and your concurrency target (the project expects 10–50 users → pool of 10–20 is typical).
- A pool that's too small causes requests to queue; too large overwhelms the database. "Pool exhausted" is a real alert condition in the project's monitoring.

19. Backup & recovery

Two complementary mechanisms:


```
# Logical backup: a re-runnable SQL dump (good for migrations, smaller DBs)
pg_dump -U app_user financial_docs > backup_2026_05_29.sql
psql -U app_user financial_docs < backup_2026_05_29.sql      # restore

# Custom compressed format + parallel restore (better for large DBs)
pg_dump -Fc -U app_user financial_docs > backup.dump
pg_restore -j 4 -d financial_docs backup.dump
```

- **WAL (Write-Ahead Log) archiving** enables **Point-In-Time Recovery (PITR)** — restore to any moment, not just the last nightly dump. The project plans nightly `pg_dump` *plus* WAL archiving.
- Backups themselves are encrypted (`gpg--symmetric`) before leaving the host, satisfying at-rest security for the financial data they contain.

Test your restores. A backup you've never restored is a hope, not a backup.

20. Security & least privilege

PostgreSQL security touchpoints in the project:

- **Least-privilege app role.** The app connects as `app_user` , which *cannot* `UPDATE` / `DELETE` `audit_logs` (§13). Schema changes happen under a separate migration/admin role.
 - **No secrets in code.** Credentials come from environment variables / a secrets store, never the source tree or Docker image.
 - **Parameterized queries.** Spring Data JPA / prepared statements prevent SQL injection — user input is *bound*, never concatenated into SQL strings.
 - **Encryption.** Data-at-rest via full-disk (LUKS) encryption on the volume; sensitive fields (e.g. bank account, tax ID) additionally encrypted at the application layer (AES-256) before storage.
 - **Network isolation.** PostgreSQL's port (5432) is not exposed to the internet — only reachable by the backend inside the Docker network.
-

21. Putting it together: the project's four tables

```
users      -- identity & RBAC: username, bcrypt password hash, role, department
documents  -- the financial record: filename, vendor, invoice_number (UNIQUE),
            -- total_amount NUMERIC, status, ocr_used, uploaded_by → users(id), embedding
audit_logs -- append-only history: user_id, action, document_id, ip_address INET,
            -- outcome, detail JSONB, created_at (no UPDATE/DELETE for app_user)
alerts     -- duplicate/fraud findings: alert_type, severity, document_ids UUID[],
            -- message, status
```

Trace a single **upload** through PostgreSQL:

1. App generates a `UUID` for the new document.
2. Inside one `@Transactional` method (`BEGIN`): `INSERT` into `documents` with `status='INDEXING'` , then `INSERT` an audit row (`DOCUMENT_UPLOAD`). `COMMIT` .
3. The `UNIQUE(invoice_number)` constraint blocks a duplicate invoice from being indexed twice.
4. After embedding/upsert to Qdrant succeeds, `UPDATE documents SET status='INDEXED'` — the `updated_at` trigger stamps the time.
5. If a duplicate is detected, `INSERT` into `alerts` with the two `document_ids` .

Every read for the UI (`GET /documents`) is a paginated, indexed `SELECT.. WHERE deleted_at IS NULL ORDER BY uploaded_at DESC LIMIT 20 OFFSET ...` , often `JOIN` ed to `users` to show the uploader's name.

22. Common pitfalls

- `UPDATE / DELETE` without `WHERE` — rewrites/erases the whole table. Write the `WHERE` first.
- `FLOAT` for money — use `NUMERIC` . Floats can't represent decimal cents exactly.
- `= NULL` — always false. Use `IS NULL` / `IS NOT NULL` .
- Forgetting `deleted_at IS NULL` — "deleted" rows reappear in lists. Use a view or partial index.
- **Over-indexing** — every index slows writes; index only what you filter/join/sort on.
- `WHERE` vs `HAVING` confusion — `WHERE` before grouping, `HAVING` after.

- **N+1 queries** — fetching documents then querying the user one-by-one in a loop; use a `JOIN`.
 - **Trusting the app for invariants** — put must-always-hold rules (uniqueness, timestamps, immutability) in constraints/triggers/privileges, not just code.
 - **`TIMESTAMP` instead of `TIMESTAMPTZ`** for audit data — loses time-zone certainty.
-

23. Practice exercises

Work these against a local Postgres (e.g. `docker run -e POSTGRES_PASSWORD=pw -p 5432:5432 postgres:15`). Solutions are intentionally not given — verify with `SELECT`.

1. Create the four project tables (`users`, `documents`, `audit_logs`, `alerts`) with correct types and constraints, including the `UNIQUE` invoice number and the FK from `documents.uploaded_by` to `users.id`.
 2. Insert 3 users (clerk, manager, admin) and 6 documents across 2 vendors. Make two documents share the *same* `invoice_number` and observe the error.
 3. Write a paginated query returning page 2 (size 3) of active documents, newest first, joined to the uploader's username.
 4. Write the fraud-detection query: per vendor, the average `total_amount` and a count, only for vendors with more than 2 documents.
 5. Add a B-tree index on `vendor` and a composite index on `(vendor, invoice_date)`. Use `EXPLAIN ANALYZE` to confirm an index scan replaces a seq scan.
 6. Wrap a document insert + audit insert in a single transaction; force a failure on the second insert and confirm the first is rolled back.
 7. Soft-delete one document, then prove it disappears from the active list but still exists in the table.
 8. Try to `DELETE` from `audit_logs` as a role with `UPDATE/DELETE` revoked and confirm the permission error.
 9. Add the `set_updated_at` trigger; update a document and confirm `updated_at` changed automatically.
 10. (Stretch) Enable `pgvector`, add an `embedding VECTOR(3)` column to a scratch table, insert 4 rows, and return the 2 nearest to a query vector by cosine distance.
-

24. Self-check questions

- Why does this project store money as `NUMERIC(15,2)` and not `FLOAT` ?
- What guarantees do the A, C, I, and D in ACID each give the upload flow?
- Why are primary keys `UUID` instead of auto-incrementing integers?
- How is the `audit_logs` table made tamper-proof, and at which layer?
- When would you choose a `JSONB` column over a normal column?
- What's the difference between soft-delete and GDPR erasure in this system?
- Why use a connection pool, and what goes wrong if it's too small or too large?
- What's the difference between `WHERE` and `HAVING` ?
- Why keep both Qdrant and a pgvector column?

If you can answer all of these in your own words, you've understood Module 1.

25. Glossary

- **DDL / DML** — Data Definition Language (structure) / Data Manipulation Language (data).
 - **ACID** — Atomicity, Consistency, Isolation, Durability — transaction guarantees.
 - **Primary key / Foreign key** — unique row identifier / reference to another table's key.
 - **Constraint** — a database-enforced rule (`NOT NULL` , `UNIQUE` , `CHECK` , FK).
 - **Index** — sorted side-structure that speeds up reads (default B-tree; GIN for JSONB; HNSW for vectors).
 - **Transaction** — a group of statements that commit or roll back together.
 - **Sequential scan** — reading every row (no usable index).
 - **Soft delete** — marking a row deleted (`deleted_at`) instead of removing it.
 - **WAL** — Write-Ahead Log; enables durability and point-in-time recovery.
 - **Connection pool** — reusable set of open DB connections (HikariCP in Spring).
 - **pgvector** — extension adding vector types and similarity search to PostgreSQL.
-

Navigation: — | [Index](#) | [Deep Dive Module 2 → Qdrant](#)